

AMATH 482/582: HOME WORK 4

EUNICE SEUNG-HYUN HAN

Department of Applied Mathematics, University of Washington, Seattle, WA
eunich@uw.edu

ABSTRACT. This study classifies images in FashionMNIST data set. Results indicate how effectively Fully Connected Deep Neural Networks with hyperparameter tuning and applying configurations was able to present high performance in classifying the fashion items of 10 classes. Figures and Tables show how the change of configurations impacts the accuracy of the FCN model. This study can be used to the Information Technology and the Artificial Intelligence industry as the classifier algorithm that predicts the fashion item by categories that could be used for enhanced search in the fashion industry.

1. Introduction and Overview

There are a lot of different kind of fashion items, including shirts, pants, and shoes. This report aims to classify images of different fashion items from the FashionMNIST data set by applying the Fully Connected Deep Neural Networks (FCNs / DNNs) as a classifier to categorize each image into one of the 10 clothing classes. The FashionMNIST data set consists 60K training images and 10K testing images while each sample is in grayscale 28×28 image. Through designing the FCN model with accurate number of training batches, number of layers, neurons in each layer, learning rate, and number of training epochs using the training loss curve and validation accuracy, the most accurate model would be applied to the test set with high prediction. By implementing the trained FCN model, the report demonstrates which model with which particular parameter value would show the highest accuracy in prediction for classifying images of different fashion items.

2. Theoretical Background

The FCN (Fully Connected Network) model is built with stacking multiple layers of neurons. When layer ℓ has n_ℓ neurons, then each of those neurons are connected to $n_{\ell-1}$ neurons from the previous layer, $(\ell - 1)$. Each layer transforms its input vector which is from the previous layer into an output vector which will eventually be the input of the next layer. The network starts with the raw data as the input layer which will flow through the hidden layers, and end with the output layer which provides the probability score distribution in classification tasks. Each layer calculates the output vector which will be the input vector for the next layer. When training the FCN model, the dataset is split into smaller sets which is the batches (selected manually). Every batch would follow the steps below through all layers. After all the batches from the entire dataset is trained, we call that one epoch is completely passed. With the selected number of epoch, the model continues training for the most accurate and high prediction.

— Every batch goes through the layers as below —

Forward pass:

$$(1) \quad z = \vec{w}^T \vec{x} + b$$

$$(2) \quad \hat{y} = \sigma(z) \implies L(\hat{y}, y)$$

- $\vec{x} \in \mathbb{R}^n$ (input vector) $\vec{w} \in \mathbb{R}^n$ (weight vector) $b \in \mathbb{R}$ (bias vector)
- $\sigma()$ is the activation function which is ReLU activation in this assignment; $\text{ReLU}(x) = \max(0, x)$

The forward pass computes the logits z in Equation (1) and then applies the ReLU activation function in Equation (2) for every layer in the FCN model. As this process repeats for all layers, the final layer ℓ , is the prediction of this network/model which provides class probabilities.

— To get the weight vector $\vec{w}$ and bias b the following equations are needed —

(3) Cross Entropy Loss:

$$L(\hat{y}, y) = -(y \log \hat{y} + (1 - y) \log(1 - \hat{y}))$$

Back propagation:

$$(4) \quad \nabla_{\vec{w}} L(\hat{y}, y) = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} \nabla_{\vec{w}} z$$

Cross Entropy Loss measure how different the predicted distribution of the model is from the true distribution/labels. By minimizing the cross entropy, it maximizes the likelihood of the prediction of the model. Then the backpropagation computes the gradient of the loss function with respect of each weight. After computing backpropagation, we need to update the parameters such as the weight and bias of the model to reduce the loss of the model and maximize the model's prediction which is the optimizers that would do that.

— Optimizers —

SGD (Stochastic Gradient Descent) [5] computes the gradients of the loss function, which is the cross entropy loss function in this assignment, with respect to the weight and bias (parameter of the model) on the batches. It updates the parameters to reduce the loss and repeats this process with many batches and epochs.

SGD Optimizer (when $J(\vec{w}, b) = L(\hat{y}(\vec{w}, b), y)$):

$$(5) \quad \vec{w} \leftarrow \vec{w} - \alpha \nabla_{\vec{w}} J(\vec{w}, b), \quad b \leftarrow b - \alpha \frac{\partial}{\partial b} J(\vec{w}, b)$$

- $\nabla_{\vec{w}} J(\vec{w}, b)$ is the gradient of J with respect to $\vec{w}$
- $\frac{\partial}{\partial b} J(\vec{w}, b)$ is the derivative of J with respect to bias, b
- α is the learning rate which controls the size of the step in the direction of the negative gradient each time when the parameters are getting updated as in Equation (5). The bigger value of α would converge fast but be unstable while smaller α would converge slow but be more stable.

The SGD updates the parameters to reduce the loss through many batches and epochs, it effectively finds the accurate parameters for the model for better prediction. As it uses small datasets, batches, instead of the entire dataset every time, it is less time consuming but still provides high accuracy. RMSProp (Root Mean Squared Propagation) [5] improves SGD by adapting the learning rate for each parameter based on the gradients of the previous history. RMSProp Optimizer:

$$(6) \quad r_{\vec{w}} \leftarrow \beta r_{\vec{w}} + (1 - \beta)(\nabla_{\vec{w}} J(\vec{w}, b))^2, \quad \vec{w} \leftarrow \vec{w} - \frac{\alpha}{\sqrt{r_{\vec{w}} + \varepsilon}} \nabla_{\vec{w}} J(\vec{w}, b)$$

$$(7) \quad r_b \leftarrow \beta r_b + (1 - \beta)\left(\frac{\partial}{\partial b} J(\vec{w}, b)\right)^2, \quad b \leftarrow b - \frac{\alpha}{\sqrt{r_b + \varepsilon}} \frac{\partial}{\partial b} J(\vec{w}, b)$$

- β is the exponential decay factor that the default is 0.9 in RMSProp
- ε is a small positive constant number that prevents division by 0
- Adapted learning rate of $\vec{w}$: $\frac{\alpha}{\sqrt{r_{\vec{w}} + \varepsilon}}$ Adapted rate of bias, b : $\frac{\alpha}{\sqrt{r_b + \varepsilon}}$

RMSProp normalizes each parameter step size which prevents the updates to not be aggressive. As this optimizer keeps track on the gradient history and uses adaptive learning rates, it can be more faster and stable than SGD. Adam (Adaptive Moment Estimation) [5] combines averaging the gradients directly ($m_{\vec{w}}$ and m_b) and averaging the squared gradients ($r_{\vec{w}}$ and r_b) which is to have smoother gradients updates with adaptive steps for the parameters. Adam Optimizer:

$$(8) \quad m_{\vec{w}} \leftarrow \beta_1 m_{\vec{w}} + (1 - \beta_1) \nabla_{\vec{w}} J(\vec{w}, b), \quad r_{\vec{w}} \leftarrow \beta_2 r_{\vec{w}} + (1 - \beta_2) (\nabla_{\vec{w}} J(\vec{w}, b))^2$$

$$(9) \quad m_b \leftarrow \beta_1 m_b + (1 - \beta_1) \frac{\partial}{\partial b} J(\vec{w}, b), \quad r_b \leftarrow \beta_2 r_b + (1 - \beta_2) \left(\frac{\partial}{\partial b} J(\vec{w}, b)\right)^2$$

Adam Optimizer first updates the first moment estimates for both weight vector and bias as in Equations (8, 9). Then update the second moment estimates by averaging the squared gradients for both weight vector and bias. Initially $m_{\vec{w}}$ and m_b can be biased toward zero and to prevent this, Adam computes:

$$(10) \quad \hat{m}_{\vec{w}} = \frac{m_{\vec{w}}}{1 - \beta_1^t}, \quad \hat{m}_b = \frac{m_b}{1 - \beta_1^t}$$

$$(11) \quad \hat{r}_{\vec{w}} = \frac{r_{\vec{w}}}{1 - \beta_2^t}, \quad \hat{r}_b = \frac{r_b}{1 - \beta_2^t}$$

- t : step count of batch iterations

Then the final parameter update for the Adam optimizer looks like:

$$(12) \quad \vec{w} \leftarrow \vec{w} - \frac{\alpha \hat{m}_{\vec{w}}}{\sqrt{\hat{r}_{\vec{w}} + \varepsilon}}, \quad b \leftarrow b - \frac{\alpha \hat{m}_b}{\sqrt{\hat{r}_b + \varepsilon}}$$

Adam optimizer helps prevent the small local minima and smooths the noisy gradients which adjusting the parameters' effective step size based on the information of the gradients. Adam combines momentum and RMSProp so it converges faster and adapts the learning rate for each weight and bias. FashionMNIST dataset contains nonlinear data so finding the optimizer through hyperparameter tunings in [Task 3](#) would effectively help to determine which optimizer performs the model to have better prediction. Training Loss is the average of the loss function from Equation (3), L , over all training samples.

$$(13) \quad \text{Training Loss} = \frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} L(\hat{y}^{(i)}, y^{(i)})$$

The validation accuracy is the the fraction of validation sample the model classifies correctly. When the model outputs the probability distribution, $\hat{p}^{(i)}$, the predicted class is selected by:

$$(14) \quad \hat{c}^{(i)} = \arg \max_c \hat{p}_c^{(i)}$$

Then the validation accuracy is calculated by:

$$(15) \quad \text{Validation Accuracy} = \frac{1}{N_{\text{val}}} \sum_{i=1}^{N_{\text{val}}} 1(\hat{c}^{(i)} = y^{(i)})$$

The indicator function, $1(\hat{c}^{(i)} = y^{(i)})$, tells when the predicted class correctly matches the true label is gives a 1 and 0 otherwise. The higher the validation accuracy is, it means there are more correctly classified validation samples. The training loss (13) and validation accuracy (15) allows to identify if the model is overfitting or underfitting which is effective in monitoring if the model is accurate. When the training metrics improves while the validation metric gets worse we can assume the model is overfitting which tells us to adjust the hyperparameters such as the learning rate. Dropout regularization [5] randomly drops a fraction of neurons from layers when training the model which helps to prevent overfitting of the training. The dropout at layer ℓ looks like:

$$(16) \quad \vec{a}_\ell^d = \vec{a}_\ell \times \frac{\vec{d}_\ell}{1 - p_d}$$

- $\vec{a}_\ell$ is the activation vector at layer ℓ
- $\vec{d}_\ell$ is the dropout mask with each entry either 1 (do not drop the neuron) or 0 (do drop the neuron)

Initialization [5] is used to set the initial values of the network's parameters such as weights and biases before the training starts. This helps prevent vanishing or exploding gradients and provide stable learning and faster convergence. Random Normal Initialization [5] uses normal distribution, $w \sim \mathcal{N}(\mu, \sigma^2)$. This is a simple technique in terms of deep learning but if σ is too large or small, it may lead to exploding or vanishing activations. Xavier Normal Initialization [5] is used for networks with tanh or sigmoid activations. It keeps the variance of the activations across layers. The Xavier Normal equation for each weight is, $w \sim \mathcal{N}(0, \frac{2}{n_{\text{in}} + n_{\text{out}}})$ when n_{in} is the number of input neurons and n_{out} is the number of output neurons of the layer. This technique works well in tanh or sigmoid networks and reduces the risk of vanishing or exploding gradients compared to normal initialization, while it is not tailored for ReLU. Kaiming (He) Uniform Initialization [5] was proposed for ReLU activations as ReLU needs a slightly different variance as it zeros out the negative inputs. Kaiming Uniform is $w \sim \text{Uniform}\left(-\sqrt{\frac{6}{n_{\text{in}}}}, \sqrt{\frac{6}{n_{\text{in}}}}\right)$. As the ReLU sets the negative inputs as zero, which halves the active neurons effectively and this technique helps to prevent vanishing and exploding in ReLU based networks.

Batch Normalization [5] normalizes the activations in each layer per each batches so each neuron has roughly mean as zero with unit variance but still have network to contain the overall scale. Batch Normalization first computes the mean of each batch by, $\mu_B = \frac{1}{B} \sum_{i=1}^B z_i$. Then it computes the variance, $\sigma_B^2 = \frac{1}{B} \sum_{i=1}^B (z_i - \mu_B)^2$. Using this mean and variance, the normalization and rescale works by, $\hat{z}_i = \frac{z_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$, $y_i = \gamma \hat{z}_i + \beta$ where y_i is the next layer's input. Batch normalization may reduce the sensitivity of the hyperparameters and is more effective when there are many layers.

3. Algorithm Implementation and Development

The algorithms and techniques that were used are FCN [5] using Cross Entropy loss (3), optimizers (RMSProp (2), Adam (12), SGD (5)), dropout regularization (16), initializations (Random Normal, Xavier Normal, Kaiming (He) Uniform), and batch normalization. The Fully Connected Neural Network composes two functions. The first function initializes the model by constructing the network layers and applying activation functions. The for loop in the first function iterates over the list of hidden layers, appending each layer and connecting it to the next with a ReLU activation [5] and then adds the output layer. The second function defines the forward pass (1) and flattens the input which goes through the network to generate the logits of each class (fashion item). Then there is another function that initializes the neural network with specific input, output, and hidden layer dimensions and trains FashionMNIST data then computes the training loss (13), validation loss, validation accuracy (15).

The function starts with the outer for-loop that trains the FCN model built ahead with the selected parameters/configurations: number of neurons for the first, hidden, last layer and the optimizer, learning rate, and epoch. The outer for-loop that runs through each epoch has an inner for-loop that processes the training batches by first setting the FCN model into training mode and resetting the gradients. Then the model would get predictions using forward pass 1, 2 and the loss function built in the FCN model function would calculate the training loss of the model with such selected configurations for each given tasks. Then the back propagation (4) is applied to calculate the gradients for the optimizer to update. After processing all batches for each epoch, the training loss for each epoch is presented. Inside the same for-loop that runs for each epoch, there is another inner for-loop that evaluates the validation batches and computes the validation loss and accuracy by comparing the model's predictions with ground-truth labels in parallel to the validation

labels. The validation accuracy (15) is determined by dividing the number of correct predictions by the total number of samples. The testing accuracy function has the same structure as the for-loop that computes the validation accuracy. As Task 2 and 3 has different parameters such the optimizer type, learning rate, initialization methods, and adjustments of batch normalization and since the structure of the FCN and the model is modular, by simply passing the parameters in the training procedure, it would provide that particular model of using specific parameters, training loss, validation loss/accuracy, and testing accuracy.

The software packages employed are NumPy [3], Matplotlib [4], Scikit-learn [6], and PyTorch [5]. NumPy [3] facilitates numerical computations such as getting the average. Matplotlib [4] provides the training loss curve and the validation accuracy curve by each epochs. Scikit-learn [6] is a machine learning library that splits the train and test data the model. PyTorch [5] is an deep learning framework that provides tools for building and training neural networks in python which includes Fully Connected Neural Network that is used to classify fashion items.

4. Computational Results

Task 1 & 2:

The configurations of the FCN model that trains (approximately 2-3 minutes) in reasonable time and results a converging training loss curve and testing accuracy that is at least above about 85% is the followings:

# of neurons: 1 st layer	# of neurons: last layer	# of hidden layers & neurons	Optimizer	Learning rate	Epoch
784	10	2 - 400 neurons, 180 neurons	SGD	0.1	20

TABLE 1. The baseline configuration/parameters of FCN model.

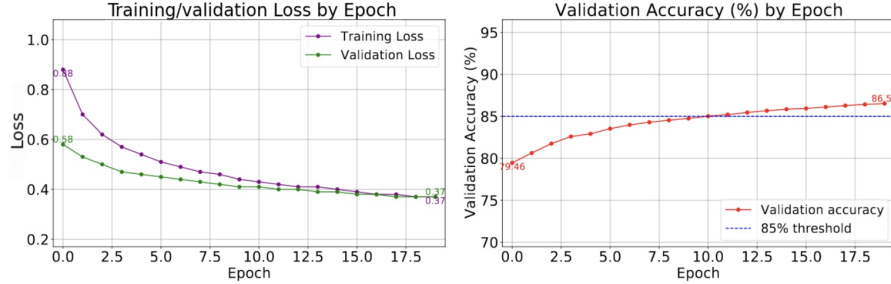


FIGURE 1. The training loss, validation loss, and validation accuracy by epoch.

Training Loss	Validation Accuracy	Testing Accuracy
0.37	86.52%	87.51%

TABLE 2. Validation/Testing accuracy using the baseline configuration from Table 1.

Figure 1 shows the training loss (13) and validation loss are converging and not showing either overfitting and underfitting in the training process. The validation accuracy (15) successfully exceeded over 85% accuracy and also Table 2 shows the validation and testing accuracy both passing over 85% accuracy which demonstrates that this FCN model and its configurations are accurate for the baseline.

Task 3 – a:

Optimizer \ LR	1e-1	1e-2	1e-3	1e-4	1e-5	1e-6	1e-7
RMSProp	731.6/22.04%	1.83/82.23%	0.26/87%	0.37/85.9%	0.7/78.58%	1.92/55.33%	2.25/15.27%
Adam	3.8/36.47%	0.28/87.52%	0.24/88.57%	0.4/86.05%	0.8/77.26%	1.9/51%	2.26/10.5%
SGD	0.36/86.52%	0.63/79.73%	1.4/59%	2.19/27.25%	2.3/10.61%	2.31/13%	2.3/8.92%

TABLE 3. Training loss / Validation accuracy of Optimizers with different Learning Rates.

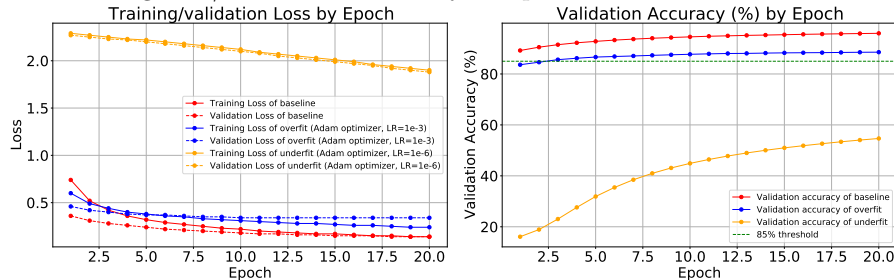


FIGURE 2. The training loss, validation loss, and validation accuracy by epoch.

Validation Accuracy	88.57%
Testing Accuracy	88.71%

TABLE 4. Validation and Testing accuracy of Adam Optimizer with learning rate = 0.001 that has the highest validation accuracy from Table 3.

Based on Table 3, the highest validation accuracy is achieved through cross validation when using the Adam Optimizer [5] with learning rate = 0.001. In Figure 2 the model that uses the Adam Optimizer with learning rate = 0.001 shows overfitting but also provides the highest validation accuracy, which tells this model would be used to get the testing accuracy in Table 4. Adam optimizer adapt the learning rate based on the gradients which performs smoother than using one global learning rate as in CGD. As Adam contains both Momentum and RMSProp, it adjusts updates more finely for each parameter which makes it more stable than RMSProp or SGD in this dataset.

Figure 2 shows the overfitting and underfitting curves of the training and validation loss. It shows it is overfitting when the learning rate=0.001 and underfitting when learning rate=0.000001 while the baseline is converging well. When the training is overfitting it means the model is trained too close to the training set which may perform poorly on new test data set and when it is underfitting it means the model did not train the patterns well so it performs poorly even in the training set. As shown in the graph, when the model overfit, the validation loss curves goes over the training loss curve which does not converge. Also when the model underfit, they are not converging and the validation accuracy is low compared to the overfit and baseline curve as it does not have enough information to get high accuracy. Table 4 shows the corresponding testing accuracy of that FCN with configurations (Adam Optimizer[5] with learning rate=0.001). Comparing it to the baseline, this shows a slightly higher accuracy which tells this configurations improved the FCN model. Task 3 – b:

Dropout Rate	0.8	0.6	0.4	0.2	0.1	0.01	0.001	0.0001
Training loss	0.64	0.5	0.42	0.4	0.38	0.37	0.37	0.36
Validation accuracy	82.41%	85.04%	85.34%	85.4%	85.66%	85.24%	85.48%	85.81%

TABLE 5. Training loss/Validation accuracy of applying Dropout regularization with different rates.

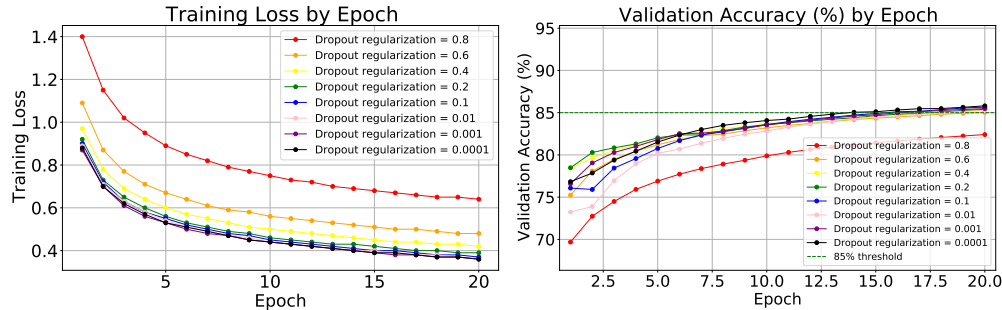


FIGURE 3. Training loss, validation loss/accuracy of applying Dropout regularization [5].

As shown in Table 5, the different dropout rates did not hugely impact the validation accuracy. In Figure3, there is no significant overfitting or underfitting. When the dropout rate is less than 0.1 then it converges and is neither over/underfitting and when it is larger than 0.1, it does not converge but may converge when it is computed with more epoch.

Validation Accuracy	85.81%
Testing Accuracy	88.3%

TABLE 6. Validation and Testing accuracy applying Dropout regularization to baseline.

The dropout rate that gave the highest validation accuracy is 0.0001 as shown in Table 5. Applying the dropout regularization, it got lower validation accuracy compared to the baseline but a slightly higher testing accuracy. Using dropout regularization (16) did improve the performance of the model, as it received higher testing accuracy than the baseline as shown in Tables 2 and 6 and it would be because the drops out prevents overreliance of specific neurons and makes the network more general with distributed representations.

Task 3 – c:

Initializations [5]	Random Normal	Xavier Normal	Kaiming (He) Uniform
Training loss/Validation accuracy	0.45/83.48%	0.32/86.17%	0.31/86.51%

TABLE 7. Training loss and Validation accuracy of applying different Initializations.

Validation Accuracy	86.51%
Testing Accuracy	86.58%

TABLE 8. Validation/Testing accuracy of applying Kaiming (He) Uniform Initialization.

Applying the Kaiming (He) Uniform Initialization [5] gave the highest validation accuracy as shown in Table 7 and by using it, the testing accuracy was received as shown in Table 8. This shows that initialization did not improve the classification compared to the baseline model 2 as the model with initialization received both lower validation and testing accuracy. It may be because the network is having only 2 layers which is shallow that can cause us to not see a large difference of using initialization methods on the baseline.

Task 3 – d:

Training Loss	Validation Accuracy	Testing Accuracy
0.23	84.77%	87.42%

TABLE 9. Training loss, Validation and Testing accuracy using Batch Normalization.

Applying the batch normalization, it gave the results shown as Table 9 which tells the testing accuracy is slightly below the baseline in Table 2. Batch normalization [5] did not improve the classification but did provide similar outputs to the baseline which means it didn't cause a significant effect on the either the training process or testing accuracy. Batch normalization works better when there are many layers [2] and as there are only 2 layers in this FCN model, batch normalization does not improve the classification.

5. Summary and Conclusions

The goal of this study was to classify images of different fashion items from the FashionMNIST data set by applying the Fully Connected Deep Neural Networks (FCNs / DNNs) as a classifier to categorize each image into one of the 10 clothing classes. The FCN model is designed with parameters/configurations (number of neurons for the first, hidden, last layer and the optimizer, learning rate, and epoch) with applying algorithms [5] and methods such as cross entropy loss(3), optimizers (RMSProp(6), Adam(12), SGD(5)), dropout regularization(16), initializations (Random Normal, Xavier Normal, Kaiming (He) Uniform), and batch normalization. Through the designed FCN model, the study was able to determine which configurations received the model that presents the highest test accuracy that converges stable. The results in Task 3-a (Table 3 , 4, Figure 2) tells the Adam optimizer [5] with learning rate = 0.001 provides the highest validation accuracy and slightly improves the baseline model by resulting with higher testing accuracy. This tells applying the Adam optimizer with learning = 0.001 effectively improves the baseline model. In Task 3-b, the dropout rate = 0.0001 gave the highest validation accuracy (15) and also improved the performance of the baseline model as shown in Table 6. In Task 3-c, Kaiming (He) Uniform Initialization gave the highest validation accuracy as shown in Table 7 but did not improve the baseline model as shown in Table 8. In Task 3-d, the batch normalization [5] did not improve the baseline model as the testing accuracy was lower as shown in Table 9.

Furthermore, this study can be extended to develop a system that classifies images of different clothing items and also other objects such as cups, plates, and chairs by using the data of 28×28 pixel for each image sample. In the future, this study could contribute valuable and enhanced resources to the Information Technology and the Artificial Intelligence industry as the classifier algorithm would predict the category of a specific item, that could be a powerful tool for automation of sorting by categories in the fashion industry.

Task 4 – Bonus:

Dataset	FashionMNIST	MNIST
Training loss/Validation accuracy	0.25/89.23%	0.06/97.67%
Testing accuracy	91.04%	98.18%

TABLE 10. Testing accuracy of > 90% for FashionMNIST and > 98% for MNIST.

The configurations that achieved the testing accuracy of > 90% for FashionMNIST and > 98% for MNIST are 4 hidden layers with each having 256, 256 ,190, 128 neurons. The dropout rate was 0.1 and the Kaiming (He) Uniform initialization [5] was used along the Adam optimizer [5] (8, 9) with learning rate 0.001 and epoch 30. By adding more layers with less neurons compared to the baseline focuses on the depth rather than the width which helps the network to learn more abstract features. Kaiming initialization [5] is effective with more layers and ReLU activations [5] so even it did not provide better testing accuracy in Task 3-c, it worked well with the new parameters/configurations under in this bonus task. Adam optimizer with learning rate 0.001 was applied with evidence from the findings in Task 3-a.

Validation Accuracy	97.37%
Testing Accuracy	96.98%

TABLE 11. Validation and Testing accuracy of KNN (number of neighbors=3), the most successful classifier from HW3, in FashionMNIST.

By comparing Table 10 and Table 11, the KNN classifier [6] provides higher validation and testing accuracy compared to the FCN model when classifying the FashionMNIST dataset. This may happen if the dataset can capture the class distributions better when using the distance measure between predictions than undertrained FCN. Also if the FCN hyperparameter tuning did not work well and presented underfitting, then the distance metric which is more straight forward works better in classifying the FashionMNIST dataset.

ACKNOWLEDGEMENTS

The author is thankful to Prof. Frank for the useful explanation about Deep Learning, Optimization, and Neural Networks as it guided the mathematical concepts and the coding layouts in the homework. The author is also thankful to the TA Rohin Gilman for going over the general information of the homework and answering specific questions for applying optimizers and initializations which helped to approach the given tasks. This report incorporates the theoretical background from the author's previous homework report [Homework 3], which expanded ideas of using mathematical concepts and different classifiers.

REFERENCES

- [1] C. da Costa-Luis, T. A. Caswell, B. Ragan-Kelley, N. Yorav-Raphael, and contributors. tqdm: A fast, extensible progress bar for python and cli. <https://github.com/tqdm/tqdm>, 2023.
- [2] N. Frank. Amath 482 A Instructor Notes. Lecture 18-22: Neural Networks, Binary Classification, Deep Learning, and Optimization.
- [3] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, Sept. 2020.
- [4] J. D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.
- [5] A. Paszke, S. Gross, S. Chintala, G. Chanan, E. Yang, Z. DeVito, Z. Lin, A. Desmaison, L. Antiga, and A. Lerer. Automatic differentiation in pytorch. In *NIPS-W*, 2017.
- [6] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, et al. Scikit-learn: Machine learning in python. *Journal of machine learning research*, 12(Oct):2825–2830, 2011.

[1][2][3][4][5][6]